

Introduction to Algorithms: 6.006

[recitation] Recitation 16: Dynamic Programming Exercises (动态规划练习)

Massachusetts Institute of Technology Instructors: Erik Demaine, Jason Ku, and Justin Solomon

MIT 6.006 Spring 2020, Recitation 16 讲义 (源 PDF:
487dc976eabd0f9bcd6a3c385203ea27_MIT6_006S20_r16.pdf, 4 页)

[exercise] Max Subarray Sum (最大子数组和)

给定一个含 n 个整数的数组 A ，任意非空 subarray (子数组) 的最大和是多少？（本课程中，subarray 总是指一段 contiguous (连续) 的元素序列）

Example (例) : $A = [-9, 1, -5, 4, 3, -6, 7, 8, -2]$ ，最大子数组和为 16。

Solution (解答) :

我们可以用 $O(n^3)$ 暴力枚举：先枚举 $O(n^2)$ 个子数组，每个用 $O(n)$ 时间求和。通过观察可以得到更快的算法——**和最大的子数组一定在某个位置结束**。计算“结束于特定位置 k 的最大子数组”可以用 $O(n)$ 时间完成：从 k 向左扫描，维护一个 rolling sum (滚动和)，沿途记住最大值；由于结束位置共有 n 个，该算法运行时间为 $O(n^2)$ 。

我们可以做得更快，只需认识到：**每一次向左扫描都在重做前面扫描已经做过的活儿**。让我们用动态规划来复用这些工作！

1. Subproblems (子问题)

- $x(k)$: 结束于 $A[k]$ 的最大子数组和
- 带条件的 prefix (前缀) 型子问题，类似 Longest Increase Subsequence (最长递增子序列)
- (练习：尝试改用 suffixes (后缀) 而不是 prefixes (前缀) 重新表述)

2. Relate (关联)

- 结束于 k 的最大子数组要么用 $k - 1$ 这个元素，要么不用
- 如果不用，那么该子数组就是单独的 $A[k]$
- 否则， $k - 1$ 被使用，我们应包含结束于 $k - 1$ 的最大子数组
- $x(k) = \max \{A[k], A[k] + x(k - 1)\}$

3. Topo. Order (拓扑序)

- 子问题 $x(k)$ 只依赖严格更小的 k ，因此无环。

4. Base (基例)

- $x(0) = A[0]$ (因为子数组必须非空)

5. Original (原问题)

- 用递归 top down 或迭代 bottom up 求解子问题
- 原问题的解是所有子问题解的最大值，即 $\max\{x(k) \mid k \in \{0, \dots, n-1\}\}$
- 每个子问题被使用两次：一次在计算下一个更大的子问题时，一次在最终求 max 时

6. Time (时间分析)

- 子问题个数: $O(n)$
- 每个子问题工作量: $O(1)$
- 求解原问题的时间: $O(n)$
- 总时间: $O(n)$

```
# bottom up implementation
def max_subarray_sum(A):
    x = [None for _ in A]          # memo
    x[0] = A[0]                   # base case
    for k in range(1, len(A)):
        x[k] = max(A[k], A[k] + x[k - 1])  # iteration / relation
    return max(x)                 # original
```

[note] Note (译者注) : 这个算法就是经典的 **Kadane's algorithm (Kadane 算法)** —— 1977 年由 **Ulf Grenander** 提出该问题 (作为数字图像中模式最大似然估计的简化模型)，Kadane 随后给出了单遍线性解法。它是"子问题带条件"的绝佳示范：朴素前缀子问题 $x(k) =$ "前 k 个元素的最大子数组和"无法直接写出递推 (新元素加入后，最大子数组可能不包含 $A[k]$ ，信息就断了)，而加上"必须结束于 $A[k]$ "这个条件后，递推只剩两种情况——要么单独取 $A[k]$ ，要么接上 $x(k-1)$ ——于是全局答案退化为对条件子问题取 max。这与你学过的 Longest Increasing Subsequence (LIS，最长递增子序列) 结构同构：LIS 的条件是"必须结束于第 i 个元素"。🐼 Demaine 在 Lecture 16 讲这类子问题设计时给出方法论："you try to ask a question whose answer would let you reduce to a smaller subproblem" (翻译：你试着提出一个问题，这个问题的答案能帮你归约到更小的子问题。)——这里的问题是" $A[k]$ 要不要并入 $x(k-1)$?" 仅此一问，两个答案，递推即成。这也是" $\max\{x(k)\}$ "这一步的意义：DP 表存的是带条件的局部最优，原问题答案要从局部最优里再取一次最大值。

[note] Note (译者注) : 讲义在这里主动点名"类似 Longest Increase Subsequence", 你恰好刚从 Lecture 16 学过 LIS 的两种解法——这个类比值得再推进一步：**为什么同样是"必须结束于某位置"的条件子问题，LIS 的朴素 DP 要 $O(n^2)$ ，Kadane 却只要 $O(n)$ ？** 答案在**转移的候选集规模**：LIS 的子序列允许"隔空取物"， $x(i)$ 可接在任意满足 $A[j] < A[i]$ 的 $x(j)$ 后面，候选集随 i 线性膨胀，每个子问题要扫 $O(n)$ 次；Kadane 的子数组必须 contiguous (连续)， $x(k)$ 只有一种接续方式——紧贴 $A[k-1]$ ——候选集恒为"单独取 $A[k]$ "与"接上 $x(k-1)$ "两项，转移 $O(1)$ ，总时间直接落到 $O(n)$ 。再把你在《计算机科学数学》9.43(e) 学过的贪心+二分搬出来对比：它没有优化朴素 DP 的转移，而是**换了一把尺子**——用单调的 tails 数组记录"长度为 k 的递增子序列的最小末尾"，二分"降低门槛"让更小的元素有机会接续（官方维基百科称之为 patience sorting）。这给出复杂度阶梯： $O(n^2)$ (LIS 朴素 DP，候选集线性) $\rightarrow O(n \log n)$ (LIS 换度量 + 单调性) $\rightarrow O(n)$ (Kadane，唯一接续)。同样是"递推需要约束、原问题负责取 max 汇总"，约束的形状决定了你能省到哪一级——连续 vs 可跳，正是 Kadane 白赚线性的根源。（顺带一提，官方维基百科给 maximum subarray problem 列出了四种解法：brute force、分治、DP、归约到最短路径——最后一种与 R15 新注 A 的"DP 即 DAG 最短路"完全呼应。）

[note] Note (译者注) : 对照代码与 SRT BOT： `x = [None for _ in A]` 是 memo (备忘录)，`x[0] = A[0]` 是 base case，循环体里的 `max(A[k], A[k] + x[k-1])` 是 relation，`return max(x)` 是 original problem 的组装——**六步法每一行都能在代码里找到落点**，这是 bottom up (自底向上) 实现的标志性特征：拓扑序就是数组下标从小到大，天然无环，连 DFS 找序都不需要（对应 Recitation 15 实现一节的说法）。注意第 5 步的提醒：每个子问题被"使用两次"（递推一次 + 最终 max 一次），这意味着**不能只记住最后一个子问题**——你必须保留整张 memo 表（或至少保留所有值以便最后取 max）。工程上若要省空间，可以滚动数组只留 $x(k-1)$ ，但那样最后一步 $\max_k x(k)$ 需要在滚动过程中同步更新——这是"空间换实现简洁性"的权衡，面试常考。

[exercise] Edit Distance (编辑距离)

一个 plagiarism detector (抄袭检测器) 需要检测两段文本 string (字符串) A 与 B 的相似度。一种相似度度量称为 edit distance (编辑距离)，即把字符串 A 变换成字符串 B 所需的最少编辑次数。一次编辑可以是三种操作之一：delete (删除) A 的一个字符；replace (替换) A 的一个字符为另一个字母；insert (插入) 一个字符到 A 的两个字符之间。描述一个 $O(|A||B|)$ 时间算法，计算 A 与 B 之间的编辑距离。

[note] Note (译者注) : Edit Distance 是 Recitation 16 的重头戏，也是算法史上最著名的 DP 应用之一，工业界无处不在：Git 的 diff 算法、拼写检查器的“你是不是想输入 XX”、DNA 序列比对（把两个基因序列的差异度量化为编辑代价）。🐼 Demaine 在 Lecture 16 讲到 LCS（最长公共子序列）时预告了这个 recitation 练习：“One version of that, you might see other versions in recitation, called edit distance.”（翻译：那是它的一个版本，你们可能会在 recitation 里看到它的其他版本，叫做编辑距离。）——**LCS 与 edit distance 是同一家族**：LCS 只允许“删除两边字符”（匹配则保留），edit distance 允许删除/插入/替换三种操作；两者的子问题都是两个字符串的前缀对（prefix pair），递推都在比较两个字符串的末尾字符。理解了一个，另一个就是“操作集合不同”的变体。这也是你大二上要学的概率论/信息论里 sequence alignment 问题的雏形。

Solution (解答) :

1. Subproblems (子问题)

- 思路：修改 A 直到它的最后一个字符与 B 匹配
- $x(i, j)$ ：把 A 到 $A(i)$ 的前缀变换成 B 到 $B(j)$ 的前缀所需的最少编辑次数
- （练习：尝试改用 suffixes（后缀）而不是 prefixes（前缀）重新表述）

2. Relate (关联)

- 如果 $A(i) = B(j)$ ，那么匹配成功！
- 否则，需要编辑使 A 的最后一个元素等于 $B(j)$
- 编辑要么是 insertion（插入）、replacement（替换）、要么是 deletion（删除）（Guess!）
- Deletion 移除 $A(i)$
- Insertion 把 $B(j)$ 加到 A 末尾，然后同时移除它与 $B(j)$
- Replacement 把 $A(i)$ 改成 $B(j)$ ，然后同时移除 $A(i)$ 与 $B(j)$

$$x(i, j) = \begin{cases} x(i-1, j-1) & \text{若 } A(i) = B(j) \\ 1 + \min(x(i-1, j), x(i, j-1), x(i-1, j-1)) & \text{否则} \end{cases}$$

3. Topo. Order (拓扑序)

- 子问题 $x(i, j)$ 只依赖严格更小的 i 和 j ，因此无环。

4. Base (基例)

- $x(i, 0) = i$, $x(0, j) = j$ （需要这么多次 insertion 或 deletion）

5. Original (原问题)

- 用递归 top down 或迭代 bottom up 求解子问题
- 原问题的解是 $x(|A|, |B|)$
- (可存储 parent pointers (父指针) 重构把 A 变换成 B 的编辑序列)

6. Time (时间分析)

- 子问题个数: $O(n^2)$
- 每个子问题工作量: $O(1)$
- 总运行时间: $O(n^2)$ (其中 $n = |A||B|$ 规模的网格, 实际为 $O(|A||B|)$)

[note] Note (译者注): ⚠ 原文此处有一处笔误: 递推式的条件写成了"若 $A(i) = B(i)$ ", 应为 $A(i) = B(j)$ ——比较的是 A 的第 i 个字符与 B 的第 j 个字符, 二者下标独立, 此处已修正。递推的直觉可以总结为一句话: **每次只盯着两个字符串的末尾字符, 做一个"对齐"决策**——若相等, 白赚一步, 直接跳过两个末尾; 若不相等, 三种操作分别对应: 删 $A(i)$ (跳 i 前进 $i - 1$)、插 $B(j)$ (B 的末尾被"消耗", j 前进 $j - 1$)、替换 (两个末尾都被消耗)。min 就是"精细的暴力枚举": 三个选项都试, 取最便宜。🧑‍🔬 关于多输入子问题空间的设计, Demaine 在 Lecture 16 给出了通用技巧: "We just take the product, multiply the subproblem spaces." (翻译: 我们直接取乘积, 把子问题空间相乘。)——两个字符串各有 $O(|A|)$ 、 $O(|B|)$ 个前缀, 乘积就是 $O(|A||B|)$ 个子问题, 每个子问题 $O(1)$ 转移, 于是 $O(|A||B|)$ 总时间。这正是"多输入问题子问题 = 各输入子问题空间的笛卡尔积"的范式。

[note] Note (译者注): 把上面的递推与你在 Lecture 16 亲手推导过的 LCS 递推并排看, 姊妹关系立即现形: **子问题同为前缀对 (i, j) , 递推都只比较末尾字符, 区别仅在操作集合**。LCS 每次决策只有两个选项——删 $A[i]$ 或删 $B[j]$ (匹配则免费保留), 在 max 意义下把匹配长度做到最大; edit distance 每次决策有三个选项——删 $A[i]$ 、插入 $B[j]$ 、把 $A[i]$ 替换成 $B[j]$, 在 min 意义下把修补代价做到最小。一个是"奖励匹配 +1 的最大化", 一个是"惩罚不匹配 +1 的最小化"——**max 与 min 的对偶**, 正如你学过的"最长路径取负即最短路径", 只是符号取向不同。两者之间还有精确数量关系: 若只允许 insert/delete (禁用 replace), 则编辑距离恰为 $|A| + |B| - 2 \cdot \text{LCS}(A, B)$ ——LCS 是已对齐的"公共骨架", A 中不在骨架的字符各删一次, B 中不在骨架的字符各插一次; 标准编辑距离因 replace 一步顶"删+插"两步而通常更小, 这正是操作集合扩展的节省。恢复方案同样同构: Lecture 16 的 LCS 用 parent pointers 重建子序列, 这里用 choice 表重建编辑序列, 与你 BFS/Dijkstra 学的父指针恢复路径是同一模式——**值表 + 回溯指针, 双表即解**。(历史注脚: 编辑距离由苏联数学家 **Vladimir Levenshtein** 于 1965 年正式定义, 官方维基百科确认其与 pairwise string alignments 密切相关; 所以它有时也直接叫 Levenshtein distance。)

[note] Note (译者注) : 注 4 引的"take the product"只是子问题空间的代数面，它还有你熟悉的几何化身：**子问题网格本身就是一张图，而且是一张 DAG**。把格子 (i, j) 当顶点——它是"A 的前 i 个字符对 B 的前 j 个字符"这个状态——三种操作的边分别从 $(i-1, j)$ 、 $(i, j-1)$ 、 $(i-1, j-1)$ 指向 (i, j) ，边权为编辑代价 0 或 1；所有边只朝下标增大方向走，行主序即天然拓扑序，于是 $x(|A|, |B|)$ 就是网格 DAG 上从 $(0, 0)$ 出发的带权最短路径长度，递推里的 min 正是你在 DAG 松弛算法中反复使用的 relaxation ($d[v] = \min\{d[u] + w(u, v)\}$)。这与你 Problem Session 7 里 Prob7-4 (Critic Collection) 学的**状态图复制**是同一思想的两面：那里把"剩余背包容量"复制成 $k+1$ 层，把带状态约束的路径问题归约回普通最短路；这里把"已处理前缀长度 (i, j) "当状态，网格天然就是状态图。区别在方向：状态图复制是**主动造图**以复用已知算法，DP 是**图本来就在、只是手工填表**——因为拓扑序已知，连建图和拓扑排序都省了。🐼 Demaine 在 Lecture 16 明说："both of these dynamic programs could instead construct a graph and then run DAG shortest paths on it. But the point is these are the same thing." (翻译：这两个动态规划都可以改为构造一张图、然后在上面跑 DAG 最短路径；关键是它们在本质上是同一回事。) ——「动态规划网格」不是比喻，它真的是一张图。(官方维基百科在 maximum subarray problem 词条里同样把"归约到最短路径"列为标准解法之一，与这条注互为印证。)

[note] Note (译者注) : 基例 $x(i, 0) = i$ 与 $x(0, j) = j$ 的含义：把长度为 i 的前缀变成空串需要 i 次删除，把空串变成长度为 j 的前缀需要 j 次插入——它们是递推的"停机条件"，保证 i 或 j 减到 0 时不再需要猜操作。**时间分析的真相：**讲义写 $O(n^2)$ 是拿 n 笼统代指两个串的长度，严格说是 $O(|A||B|)$ ——子问题网格是 $|A| \times |B|$ 。面试中把 $|A|, |B|$ 说清楚，比笼统的 $O(n^2)$ 更能体现理解深度。

```
def edit_distance(A, B):
    x = [[None] * len(A) for _ in range(len(B))] # memo
    x[0][0] = 0 # base cases
    for i in range(1, len(A)):
        x[i][0] = x[i-1][0] + 1 # delete A[i]
    for j in range(1, len(B)):
        x[0][j] = x[0][j-1] + 1 # insert B[j] into A
    for i in range(1, len(A)): # dynamic program
        for j in range(1, len(B)):
            if A[i] == B[j]:
                x[i][j] = x[i-1][j-1] # matched! no edit needed
            else: # edit needed!
                ed_del = 1 + x[i-1][j] # delete A[i]
                ed_ins = 1 + x[i][j-1] # insert B[j] after A[i]
                ed_rep = 1 + x[i-1][j-1] # replace A[i] with B[j]
                x[i][j] = min(ed_del, ed_ins, ed_rep)
    return x[len(A)-1][len(B)-1]
```

[note] Note (译者注) : 这个 bottom up 实现有三处值得细读。其一，**基例被单独成环先填**：先填第 0 行 ($x[i][0] = i$) 与第 0 列 ($x[0][j] = j$)，主体 DP 从 (1, 1) 开始——讲义在文末练习题里点明这样设计是为了让"重构编辑序列"更容易（重建时可以从表的右下角沿着父指针回溯，遇到基例即停）。其二，注意代码用 `len(A)` 建行、`len(B)` 建列，与递推式中 $x(i, j)$ 的语义完全对应，但若 A 或 B 为空串，`x[0][0]` 与下标访问会越界——严谨的实现应先用基例特判空串。其三，对照 Recitation 15 的 parent pointers 讨论：这段代码只返回距离值；要返回编辑序列，需在每次选择 min 时记录"选了哪个操作"（或存父指针三元组），最后回溯——这正是文末练习的目标。

Exercise (练习) : 修改上面的代码，返回把字符串 A 变换成字符串 B 的最少编辑序列。（注意，上面代码中的基例是逐个计算的，目的是让重构解更容易。）

[note] Note (译者注) : 练习提示已经很直白：基例单独填的目的就是方便回溯时识别"走到了哪个边界"。实现思路：并行维护一个 `choice` 表（大小与 memo 相同），在每次 `x[i][j] = min(...)` 时记录 0/1/2（对应替换/删除/插入或匹配），最后从 $(|A|, |B|)$ 倒着走回 (0, 0)，把操作序列逆序输出。这与你学过的 shortest path 重构完全同构：DP 表 = 距离表，choice 表 = 父指针。工业界的 Smith–Waterman / Needleman–Wunsch 序列比对算法正是这一模式在生物信息学中的直接应用——本题的 $O(1)$ 额外空间技巧（用 choice 表而非满父指针）在那里同样重要。

[note] Note (译者注) : 编辑距离是你"大二上"两门课的提前彩排，先把脚手架搭好，将来它们回访这个算法时你能认出来。**信息论 (Cover 《Elements of Information Theory》)** 研究"两个序列在什么意义上相似"，编辑距离正是把"相似"量化成"最小变换代价"的尺子；你在 Cover 里会遇到同类尺子——信道编码中衡量两个码字差异的 Hamming distance（汉明距离，逐位比较），以及率失真理论中的失真度量。区别要看清：Hamming distance 要求逐位对齐，编辑距离允许删、插、换造成的错位，因此能处理"基因序列里插进了一段"这类比对，这正是 DNA/蛋白质序列比对（Smith–Waterman 算法）选用它的原因。**概率论**

（Bertsekas 《概率导论》）的连接是：把递推里的 min 换成 max、把固定代价换成概率（或负对数概率），编辑距离的骨架就变成"找最可能的对齐路径"——这正是语音识别中 Viterbi 算法（官方维基百科确认：在 hidden Markov models (HMM, 隐马尔可夫模型) 上找最可能隐藏状态序列的动态规划）的递推。两者分叉点：概率论关注随机序列的**期望**代价（按概率加权平均），Viterbi 关注**概率最大**的那条路径（取 max 而非平均），你大二上学的期望与条件概率就是读懂 Viterbi 的引桥。**这一条信息量大，记不住细节没关系——只需记住一句话：编辑距离的 DP 骨架（前缀对 + 末尾字符决策 + min/max 聚合）是信息论、概率论、优化共同的地基。**

Recitation 16 结束。译者注 11 条：Kadane 条件子问题、Kadane $\leftrightarrow$ LIS 复杂度阶梯（候选集规模）、bottom up 代码与 SRT BOT 对照、LCS/edit distance 家族、递推直觉与原文笔误修正、edit $\leftrightarrow$ LCS 对偶与公式（Levenshtein 1965）、网格=DAG=最短路（状态图复制同构）、基例与时间分析、bottom up 实现细节、parent pointers 重构、信息论/概率论预告（Viterbi）。

[reference] Reference (来源与延伸)

- MIT 6.006 Spring 2020, Recitation 16 (本讲义)
- Lecture 16: Dynamic Programming, Part 2 — LCS, LIS, Coins (视频字幕已引用)
- CLRS 4th ed. §14.4 (Longest Common Subsequence, 与编辑距离同源); J. B. Kruskal 1983, *An Overview of Sequence Comparison*
- 官方维基百科 (2026-06 离线版): Maximum subarray problem (Grenander 1977 提出, Kadane 单遍算法, 四种解法含归约到最短路径)、Levenshtein distance (1965)、Viterbi algorithm (HMM)、Dynamic programming (Bellman 1950s)
- 关联讲义: `lec16-dyn-prog-subproblems.bilingual.md` (LCS 递推、product 与 DAG 最短路径等价、parent pointers 重建)、`r15.bilingual.md` (SRT BOT 框架)、`prob7sol.bilingual.md` (状态图复制, Prob7-4)

MIT OpenCourseWare — <https://ocw.mit.edu> — 6.006 Introduction to Algorithms, Spring 2020.
Terms of Use: <https://ocw.mit.edu/terms>